

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI – 602105
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 –Dataset Intégration Task

Course Code:	DSA0502	Course Title:	Query Processing for Data Science
CO Assessed:	CO3	Assessment:	Tool 3 - Dataset Intégration Task
Weightage:		Total Marks:	
CO3 Statement: Identify and clean inconsistencies in data by handling missing values, duplicates, outliers, formatting issues, and applying techniques like fuzzy matching and regex. (BL5 and BL6).			
Student Name:	Dharshini H	Reg. No.:	192324268
Section / Batch:	B.Tech AI and DS	Date of Submission:	16-08-2026
Faculty In-charge:	Dr. B.Shyamala Gowri	Project Mode:	Individual Submission

Code of Conduct

I Dharshini H, 192324268 (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: Dharshini H

Dataset Intégration Task

Question 1 – Customer Dataset Integration

Three customer datasets from **Online, Retail, and Support** systems contain inconsistent names, phone numbers, email addresses, dates, duplicate records, and missing values.

Task: Using Python/Pandas, clean and integrate the datasets by identifying invalid values, formatting data, removing duplicates, applying fuzzy matching and RegEx validation, standardizing fields, and saving the final master dataset in CSV and JSON formats. Create a reusable cleanup script and test it with a new dataset.

CO3 > at3_1.py > test_new_dataset

```
1 import pandas as pd
2 import re
3 from rapidfuzz import fuzz
4 import os
5 # -----
6 # CREATE OUTPUT FOLDER
7 # -----
8 os.makedirs("output", exist_ok=True)
9 # -----
10 # LOAD DATASET
11 # -----
12 def load_dataset(file_path, source):
13     df = pd.read_csv(file_path)
14     df.columns = (
15         df.columns
16         .str.strip()
17         .str.lower()
18         .str.replace(" ", "_")
19     )
20     df["source"] = source
21     return df
22 # -----
23 # CLEAN NAME
24 # -----
25 def clean_name(name):
26     if pd.isna(name):
27         return None
28     name = str(name).strip()
```

CO3 > at3_1.py > test_new_dataset

```
25 def clean_name(name):
26     if pd.isna(name):
27         return None
28     name = str(name).strip()
29     # Remove extra spaces
30     name = re.sub(r"\s+", " ", name)
31     # Keep only alphabets and spaces
32     name = re.sub(r"[^A-Za-z\s]", "", name)
33     # Title case
34     name = name.title()
35     return name
36
37 # -----
38 # CLEAN PHONE
39 # -----
40 def clean_phone(phone):
41     if pd.isna(phone):
42         return None
43     phone = str(phone)
44     # Remove spaces, +, -, brackets, etc.
45     phone = re.sub(r"\D", "", phone)
46     # Remove Indian country code
47     if phone.startswith("91") and len(phone) == 12:
48         phone = phone[2:]
49     # Validate 10 digit number
50     if len(phone) == 10:
51         return phone
52     return None
```

CO3 > at3_1.py > test_new_dataset

```
53 # -----
54 # VALIDATE EMAIL USING REGEX
55 # -----
56 def clean_email(email):
57     if pd.isna(email):
58         return None
59     email = str(email).strip().lower()
60     pattern = r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"
61     if re.match(pattern, email):
62         return email
63     return None
64
65 # -----
66 # CLEAN DATE
67 # -----
68 def clean_date(date):
69     if pd.isna(date):
70         return None
71     result = pd.to_datetime(
72         date,
73         errors="coerce",
74         dayfirst=True
75     )
76     if pd.isna(result):
77         return None
78     return result.strftime("%Y-%m-%d")
79
80 # -----
81 # CLEAN COMPLETE DATASET
82 # -----
83 def clean_dataset(df):
84     df["name"] = df["name"].apply(clean_name)
85     df["phone"] = df["phone"].apply(clean_phone)
86     df["email"] = df["email"].apply(clean_email)
87     df["date"] = df["date"].apply(clean_date)
88     # Convert empty values to NaN
89     df = df.replace(r"^\s*$", pd.NA, regex=True)
90     # Remove exact duplicate rows
91     df = df.drop_duplicates()
92     return df
93
94 # FUZZY MATCHING
95 # -----
96 def similarity(name1, name2):
97     if pd.isna(name1) or pd.isna(name2):
98         return 0
99     return fuzz.ratio(
100         str(name1).lower(),
101         str(name2).lower()
102     )
```

CO3 > at3_1.py > test_new_dataset

```
103 # -----
104 # FIND MASTER CUSTOMER ID
105 # -----
106 def assign_master_ids(df):
107     master_records = []
108     master_ids = []
109     next_id = 1
110     for _, row in df.iterrows():
111         name = row["name"]
112         phone = row["phone"]
113         email = row["email"]
114         matched_id = None
115         for record in master_records:
116             name_score = similarity(
117                 name,
118                 record["name"]
119             )
120             phone_match = (
121                 phone is not None
122                 and record["phone"] is not None
123                 and phone == record["phone"]
124             )
125             email_match = (
126                 email is not None
127                 and record["email"] is not None
128                 and email == record["email"]
129             )
```

```

130         # Same phone/email OR highly similar name
131         if (
132             phone_match
133             or email_match
134             or name_score >= 85
135         ):
136             matched_id = record["master_id"]
137             break
138         if matched_id is None:
139             matched_id = f"M{next_id:04d}"
140             master_records.append({
141                 "master_id": matched_id,
142                 "name": name,
143                 "phone": phone,
144                 "email": email
145             })
146             next_id += 1
147             master_ids.append(matched_id)
148         df["MasterCustomerID"] = master_ids
149         return df

150 # -----
151 # INTEGRATE DATASETS
152 # -----
153 def integrate_datasets():
154     online = load_dataset(
155         r"C:\Users\dhars\Documents\Query processing\lab_questions\C03\online.csv",
156         "Online"
157     )
158     retail = load_dataset(
159         r"C:\Users\dhars\Documents\Query processing\lab_questions\C03\retail.csv",
160         "Retail"
161     )
162     support = load_dataset(
163         r"C:\Users\dhars\Documents\Query processing\lab_questions\C03\support.csv",
164         "Support"
165     )
166     # Clean each dataset
167     online = clean_dataset(online)
168     retail = clean_dataset(retail)
169     support = clean_dataset(support)
170     # Combine datasets
171     combined = pd.concat(
172         [online, retail, support],
173         ignore_index=True
174     )
175     return combined

176 # -----
177 # SAVE OUTPUT
178 # -----
179 def save_output(df):
180     csv_path = "output/master_customer.csv"
181     json_path = "output/master_customer.json"
182     df.to_csv(
183         csv_path,
184         index=False
185     )
186     df.to_json(
187         json_path,
188         orient="records",
189         indent=4
190     )
191     print("\nOutput files created:")
192     print(csv_path)
193     print(json_path)
194 # -----
195 # TEST NEW DATASET
196 # -----
197 def test_new_dataset():
198     print("\nTesting reusable script with new dataset...")
199     test = load_dataset(
200         r"C:\Users\dhars\Documents\Query processing\lab_questions\C03\new_test_data.csv",
201         "NewTest"
202     )
203     test = clean_dataset(test)
204     test = assign_master_ids(test)

```

```

194 # -----
195 # TEST NEW DATASET
196 # -----
197 def test_new_dataset():
198     print("\nTesting reusable script with new dataset...")
199     test = load_dataset(
200         "C:\\Users\\dhars\\Documents\\Query processing\\lab_questions\\C03\\new_test_data.csv",
201         "NewTest"
202     )
203     test = clean_dataset(test)
204     test = assign_master_ids(test)
205     print("\nNew Dataset After Cleaning:")
206     print(test.to_string(index=False))

```

```

210 def main():
211     print("=====")
212     print(" CUSTOMER DATA INTEGRATION SYSTEM")
213     print("=====")
214     # Integrate three datasets
215     df = integrate_datasets()
216     print("\nTotal records after cleaning:")
217     print(len(df))
218     # Assign master customer IDs
219     df = assign_master_ids(df)
220     # Remove duplicate customer records
221     df = df.drop_duplicates(
222         subset=[
223             "MasterCustomerID",
224             "name",
225             "phone",
226             "email"
227         ]
228     )
229     # Save final master dataset
230     save_output(df)
231     print("\nFinal Master Dataset:")
232     print(df.to_string(index=False))
233     # Test reusable script
234     test_new_dataset()
235 # RUN PROGRAM
236 if __name__ == "__main__":
237     main()

```

OUTPUT:

```
Total records after cleaning:
12
```

```
Output files created:
output/master_customer.csv
output/master_customer.json
```

```
Final Master Dataset:
customerid      name      phone      email      date source MasterCustomerID
C001  John Smith  9876543210  john.smith@gmail.com  2026-10-01 Online      M0001
C002  Priya Sharma  9876543211  priya.sharma@gmail.com  2026-01-10 Online      M0002
C003  Rahul Kumar  9876543212      rahul@gmail.com  2026-02-15 Online      M0003
C004  Anita Rao  9876543213      anita@gmail.com  2026-02-15 Online      M0004
R102  Priya S  9876543211  priya.sharma@gmail.com  2026-10-01 Retail      M0002
R104  Anitha Rao  9876543213      anita@gmail.com  2026-02-15 Retail      M0004
```

```
Testing reusable script with new dataset...
c:\Users\dhars\Documents\Query processing\lab_questions\C03\at3_1.py:71: UserWarning: Parsing dates in %Y/%m/%d format when dayfirst=True was specified. Pass `dayfirst=False` or specify a format to silence this warning.
  result = pd.to_datetime(
```

```
New Dataset After Cleaning:
customerid      name      phone      email      date source MasterCustomerID
N001  John Smith  9876543210  john.smith@gmail.com  2026-08-16 NewTest      M0001
N002  Anitha Rao  9876543213      anita@gmail.com  2026-08-16 NewTest      M0002
N003  New Customer  9876543299  newcustomer@gmail.com  2026-08-16 NewTest      M0003
N004  Priya Sharma  9876543211  priya.sharma@gmail.com  2026-08-16 NewTest      M0004
N005  Wrong Email  9876543300      NaN  2026-08-16 NewTest      M0005
```

PS C:\Users\dhars\Documents\Query processing\lab_questions> |

Question 2 – Employee Dataset Integration

An organization has employee data stored in **HR, Payroll, and Attendance datasets**. The datasets contain different column names, inconsistent employee names, duplicate employees, invalid email IDs, different date formats, missing department values, and salary inconsistencies.

Task: Develop a Python program to integrate the three datasets. Apply data formatting, RegEx matching, duplicate detection, fuzzy matching, normalization/standardization, outlier detection, and missing-value handling. Save the cleaned integrated dataset and test the cleanup script using new employee records.

```
1  import pandas as pd
2  import re
3  from rapidfuzz import fuzz
4  import os
5  # 1. LOAD DATA
6  def load_dataset(file_path, source):
7      df = pd.read_csv(file_path)
8      # Standardize column names
9      df.columns = (
10         df.columns
11         .str.strip()
12         .str.lower()
13         .str.replace(" ", "_")
14     )
15     df["source"] = source
16     return df
17
18 # 2. STANDARDIZE NAME
19 def clean_name(name):
20     if pd.isna(name):
21         return None
22     name = str(name).strip()
23     # Remove extra spaces
24     name = re.sub(r"\s+", " ", name)
25     # Remove unwanted characters
26     name = re.sub(r"[^A-Za-z\s]", "", name)
27     return name.title()
28
29 # 3. EMAIL VALIDATION USING REGEX
30 def clean_email(email):
31     if pd.isna(email):
32         return None
33     email = str(email).strip().lower()
34     pattern = r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"
35     if re.match(pattern, email):
36         return email
37     return None
38
39 # 4. STANDARDIZE DEPARTMENT
40 def clean_department(department):
41     if pd.isna(department):
42         return None
43     department = str(department).strip().lower()
44     department = re.sub(r"\s+", " ", department)
45     department_mapping = {
46         "it": "Information Technology",
47         "information technology": "Information Technology",
48         "hr": "Human Resources",
49         "human resources": "Human Resources",
50         "finance": "Finance",
51         "marketing": "Marketing"
52     }
53     return department_mapping.get(
54         department,
55         department.title()
```

```

58 # 5. STANDARDIZE DATE
59 def clean_date(date):
60     if pd.isna(date):
61         return None
62     date = str(date).strip()
63     # YYYY-MM-DD / YYYY/MM/DD
64     if re.match(r"^\d{4}[-/]\d{2}[-/]\d{2}$", date):
65         result = pd.to_datetime(
66             date,
67             errors="coerce",
68             yearfirst=True
69         )
70     # DD-MM-YYYY / DD/MM/YYYY
71     else:
72         result = pd.to_datetime(
73             date,
74             errors="coerce",
75             dayfirst=True
76         )
77     if pd.isna(result):
78         return None
79     return result.strftime("%Y-%m-%d")
80
81 # 6. CLEAN SALARY
82 def clean_salary(salary):
83     if pd.isna(salary):
84         return None
85     try:
86         salary = float(
87             str(salary)
88             .replace(".", "")
89             .replace("₹", "")
90         )
91     except:
92         return None

```

```

93
94 # 7. CLEAN DATASET
95 def clean_dataset(df):
96     # Rename different columns
97     rename_columns = {
98         "empid": "employee_id", "employee_id": "employee_id",
99         "employee_name": "name", "name": "name",
100         "dept": "department", "department": "department",
101         "joiningdate": "joining_date",
102         "joining_date": "joining_date",
103         "paymentdate": "payment_date",
104         "payment_date": "payment_date",
105         "attendancedate": "attendance_date",
106         "attendance_date": "attendance_date"
107     }
108     df = df.rename(columns=rename_columns)
109     # Clean fields
110     if "name" in df.columns:
111         df["name"] = df["name"].apply(clean_name)
112     if "email" in df.columns:
113         df["email"] = df["email"].apply(clean_email)
114     if "department" in df.columns:
115         df["department"] = df["department"].apply(
116             clean_department
117         )
118     if "joining_date" in df.columns:
119         df["joining_date"] = df["joining_date"].apply(
120             clean_date
121         )
122     if "payment_date" in df.columns:
123         df["payment_date"] = df["payment_date"].apply(
124             clean_date
125         )
126     if "attendance_date" in df.columns:
127         df["attendance_date"] = df["attendance_date"].apply(
128             clean_date
129         )
130     if "salary" in df.columns:
131         df["salary"] = df["salary"].apply(
132             clean_salary
133         )
134     df = df.drop_duplicates()
135     return df

```

```

137 # 8. FUZZY MATCHING
138 def name_similarity(name1, name2):
139     if pd.isna(name1) or pd.isna(name2):
140         return 0
141     return fuzz.ratio(
142         str(name1).lower(),
143         str(name2).lower()
144     )
145

```

```

146 # 9. DETECT SALARY OUTLIERS
147 def detect_salary_outliers(df):
148     if "salary" not in df.columns:
149         return df
150     salary = df["salary"]
151     Q1 = salary.quantile(0.25)
152     Q3 = salary.quantile(0.75)
153     IQR = Q3 - Q1
154     lower_limit = Q1 - 1.5 * IQR
155     upper_limit = Q3 + 1.5 * IQR
156     df["salary_outlier"] = (
157         (salary < lower_limit) |
158         (salary > upper_limit)
159     )
160     return df

```

```

162 # 10. ASSIGN MASTER EMPLOYEE ID
163 def assign_master_ids(df):
164     master_records = []
165     master_ids = []
166     next_id = 1
167     for _, row in df.iterrows():
168         name = row.get("name")
169         email = row.get("email")
170         employee_id = row.get("employee_id")
171         matched_id = None
172         for record in master_records:
173             name_score = name_similarity(
174                 name,
175                 record["name"]
176             )
177             email_match = (
178                 email is not None
179                 and record["email"] is not None
180                 and email == record["email"]
181             )
182             employee_match = (
183                 employee_id is not None
184                 and record["employee_id"] is not None
185                 and employee_id == record["employee_id"]
186             )
187             if (
188                 email_match
189                 or employee_match
190                 or name_score >= 85
191             ):
192                 matched_id = record["master_id"]
193                 break
194             if matched_id is None:
195                 matched_id = f"ME{next_id:04d}"
196                 master_records.append({
197                     "master_id": matched_id, "name": name,
198                     "email": email,
199                     "employee_id": employee_id
200                 })
201                 next_id += 1
202             master_ids.append(matched_id)
203         df["MasterEmployeeID"] = master_ids
204     return df
205
206 # 11. INTEGRATE HR, PAYROLL, ATTENDANCE
207 def integrate_datasets():
208     base_path = r"C:\Users\dhars\Documents\Query processing\lab_questions\C03"
209     hr = load_dataset(
210         os.path.join(base_path, r"C:\Users\dhars\Documents\Query processing\lab_questions\C03\hr.csv"),
211         "HR"
212     )
213     payroll = load_dataset(
214         os.path.join(base_path, r"C:\Users\dhars\Documents\Query processing\lab_questions\C03\payroll.csv"),
215         "Payroll"
216     )
217     attendance = load_dataset(
218         os.path.join(base_path, r"C:\Users\dhars\Documents\Query processing\lab_questions\C03\attendance.csv"),
219         "Attendance"
220     )
221     hr = clean_dataset(hr)
222     payroll = clean_dataset(payroll)
223     attendance = clean_dataset(attendance)
224     combined = pd.concat(
225         [
226             hr,
227             payroll,
228             attendance
229         ],
230         ignore_index=True,
231         sort=False
232     )
233     return combined
234
235 # 12. MISSING VALUE HANDLING
236 def handle_missing_values(df):
237     # Department
238     if "department" in df.columns:
239         df["department"] = df["department"].fillna(
240             "Unknown"
241         )
242     # Email
243     if "email" in df.columns:
244         df["email"] = df["email"].fillna(
245             "Not Available"
246         )
247     # Salary
248     if "salary" in df.columns:
249         df["salary"] = df["salary"].fillna(
250             df["salary"].median()
251         )
252     return df
253
254 # 13. SAVE DATA
255 def save_output(df):
256     output_folder = r"C:\Users\dhars\Documents\Query processing\lab_questions\C03\output"
257     os.makedirs(
258         output_folder,
259         exist_ok=True
260     )
261     csv_path = os.path.join(
262         output_folder,
263         "master_employee.csv"
264     )
265     json_path = os.path.join(
266         output_folder,
267         "master_employee.json"
268     )
269     df.to_csv(
270         csv_path,
271         index=False
272     )
273     df.to_json(
274         json_path,
275         orient="records",
276         indent=4
277     )

```



```

282 # 14. TEST NEW DATASET
283 def test_new_dataset():
284     print("\n=====")
285     print(" TESTING NEW EMPLOYEE DATA")
286     print("=====")
287     path = (
288         r"C:\Users\dhars\Documents\Query processing\lab_questions\CO3\new_employee.csv"
289     )
290     test = load_dataset(
291         path,
292         "NewRecords"
293     )
294     test = clean_dataset(test)
295     test = assign_master_ids(test)
296     test = detect_salary_outliers(test)
297     test = handle_missing_values(test)
298     print("\nCleaned New Employee Records:")
299     print(
300         test.to_string(index=False)
301     )
302 # 15. MAIN
303 def main():
304     print("=====")
305     print(" EMPLOYEE DATA INTEGRATION SYSTEM")
306     print("=====")
307     df = integrate_datasets()
308     print(
309         "\nTotal records after cleaning:",
310         len(df)
311     )
312     df = assign_master_ids(df)
313     df = detect_salary_outliers(df)
314     df = handle_missing_values(df)
315     df = df.drop_duplicates(
316         subset=[
317             "MasterEmployeeID",
318             "name",
319             "email"
320         ]
321     )
322     save_output(df)
323     print("\nFinal Integrated Dataset:")
324     print(
325         df.to_string(index=False)
326     )
327     test_new_dataset()
328 if __name__ == "__main__":
329     main()

```

OUTPUT:

Total records after cleaning: 30

Files created:
C:\Users\dhars\Documents\Query processing\lab_questions\CO3\output\master_employee.csv
C:\Users\dhars\Documents\Query processing\lab_questions\CO3\output\master_employee.json

Final Integrated Dataset:

employeeid	employeename	email	department	joining_date	salary	source	employee_id	name	payment_date	attendance_date	status	MasterEmployeeID	salary_outlier
E001	Arun Kumar	arun.kumar@gmail.com	Information Technology	2024-01-10	55000.0	HR	NaN	NaN	NaN	NaN	NaN	ME0001	False
E002	Priya Sharma	priya.sharma@gmail.com	Human Resources	2024-02-15	60000.0	HR	NaN	NaN	NaN	NaN	NaN	ME0002	False
E003	Rahul Kumar	rahul.kumar@gmail.com	Finance	2024-03-20	58000.0	HR	NaN	NaN	NaN	NaN	NaN	ME0003	False
E004	Anita Rao	anita.rao@gmail.com	Information Technology	2024-04-05	62000.0	HR	NaN	NaN	NaN	NaN	NaN	ME0004	False
E005	Sneha Patel	sneha.patel@gmail.com	Marketing	2024-05-10	52000.0	HR	NaN	NaN	NaN	NaN	NaN	ME0005	False
E006	Vijay Kumar	vijay.kumar@gmail.com	Finance	2024-06-15	59000.0	HR	NaN	NaN	NaN	NaN	NaN	ME0006	False
E007	Meena Devi	meena.dev@gmail.com	Human Resources	2024-06-20	61000.0	HR	NaN	NaN	NaN	NaN	NaN	ME0007	False
E008	Krishna Raj	krishna.raj@gmail.com	Information Technology	2024-07-25	65000.0	HR	NaN	NaN	NaN	NaN	NaN	ME0008	False
E009	Neha Singh	neha.singh@gmail.com	Marketing	2024-08-10	54000.0	HR	NaN	NaN	NaN	NaN	NaN	ME0009	False
E010	Ravi Sharma	ravi.sharma@gmail.com	Finance	2024-08-15	57000.0	HR	NaN	NaN	NaN	NaN	NaN	ME0010	False
NaN	NaN	arun.kumar@gmail.com	Information Technology	NaN	55000.0	Payroll	E001	Arun Kumar	2026-01-31	NaN	NaN	ME0001	False
NaN	NaN	priya.sharma@gmail.com	Human Resources	NaN	60000.0	Payroll	E002	Priya S	2026-01-31	NaN	NaN	ME0002	False
NaN	NaN	rahul.kumar@gmail.com	Finance	NaN	58000.0	Payroll	E003	Rahul Kumar	2026-01-31	NaN	NaN	ME0003	False
NaN	NaN	anita.rao@gmail.com	Information Technology	NaN	62000.0	Payroll	E004	Anitha Rao	2026-01-31	NaN	NaN	ME0004	False
NaN	NaN	sneha.patel@gmail.com	Marketing	NaN	52000.0	Payroll	E005	Sneha Patel	2026-01-31	NaN	NaN	ME0005	False
NaN	NaN	vijay.kumar@gmail.com	Finance	NaN	59000.0	Payroll	E006	Vijay Kumar	2026-01-31	NaN	NaN	ME0006	False
NaN	NaN	meena.dev@gmail.com	Human Resources	NaN	61000.0	Payroll	E007	Meena Devi	2026-01-31	NaN	NaN	ME0007	False
NaN	NaN	krishna.raj@gmail.com	Information Technology	NaN	150000.0	Payroll	E008	Krishna Raj	2026-01-31	NaN	NaN	ME0008	True
NaN	NaN	Not Available	Marketing	NaN	54000.0	Payroll	E009	Neha Singh	2026-01-31	NaN	NaN	ME0011	False
NaN	NaN	ravi.sharma@gmail.com	Finance	NaN	57000.0	Payroll	E010	Ravi Sharma	2026-01-31	NaN	NaN	ME0010	False
NaN	NaN	anita.rao@gmail.com	Unknown	NaN	58500.0	Attendance	E002	Priya Sharma	NaN	2026-08-01	Present	ME0002	False
NaN	NaN	neha.singh@gmail.com	Information Technology	NaN	58500.0	Attendance	E004	Anita Rao	NaN	2026-08-01	Present	ME0004	False
NaN	NaN	neha.singh@gmail.com	Unknown	NaN	58500.0	Attendance	E009	Neha Singh	NaN	2026-08-01	Absent	ME0009	False

=====

TESTING NEW EMPLOYEE DATA

=====

Cleaned New Employee Records:

employeeid	employeename	email	department	joining_date	salary	source	MasterEmployeeID	salary_outlier
M001	Arun Kumar	arun.kumar@gmail.com	Information Technology	2026-08-15	55000.0	NewRecords	ME0001	False
M002	Anitha Rao	anita.rao@gmail.com	Information Technology	2026-08-16	62000.0	NewRecords	ME0002	False
M003	New Employee	newemployee@gmail.com	Information Technology	2026-08-17	58000.0	NewRecords	ME0003	False
M004	Priya S	Not Available	Unknown	2026-08-18	200000.0	NewRecords	ME0004	True

PS C:\Users\dhars\Documents\Query processing\lab_questions> |